EXPERIMENT 9

Shravanya Andhale

D20A-02

Experiment 9: Retrieval-Augmented Generation (RAG) Based Question Answering System

AIM:

To develop a Retrieval-Augmented Generation (RAG) application for document-based question answering by retrieving relevant document chunks from a vector store and supplying the retrieved context to a generative language model to produce a grounded answer.

THEORY:

Retrieval-Augmented Generation (RAG) is a Generative AI architecture that combines information retrieval with text generation. It allows a Large Language Model (LLM) to generate answers using information retrieved from an external document collection rather than depending only on the knowledge stored in its model parameters.

A normal LLM generates responses based on patterns and information learned during training. However, it may not have access to specific documents, updated information, or private domain knowledge. It may also generate incorrect or unsupported information, commonly known as hallucinations. RAG addresses these limitations by retrieving relevant information from a selected knowledge source and providing it to the model as additional context.

1. Document Ingestion:

The first stage of a RAG system is document ingestion. Documents such as PDF, TXT, or DOCX files are loaded into the application. Their text is extracted and processed to remove unnecessary formatting noise while retaining meaningful content. The extracted text forms the knowledge source of the application.

2. Document Chunking:

Large documents are divided into smaller sections called chunks. Chunking makes it easier to retrieve specific information related to a question. Each chunk can contain

approximately 100 to 300 words, depending on the document structure. Unique chunk IDs and metadata such as document title, source, page number, and section are maintained for identification.

3. Embedding Generation:

An embedding model converts each document chunk into a numerical vector. These vectors represent the semantic meaning of the text. The embeddings allow the system to identify chunks that are related to a user's question even when the wording is different.

4. Vector Store and Retrieval:

The generated embeddings are stored in a vector database or index such as FAISS or ChromaDB. When a user submits a question, the same embedding model converts the query into a vector. The system compares the query embedding with the stored document embeddings and retrieves the top-k most relevant chunks using similarity search.

5. Context Construction:

The retrieved chunks are combined with the user's original question to form a prompt for the generative model. The prompt clearly separates the retrieved context from the question. It can instruct the model to answer only using the supplied information and state when the available documents do not contain enough information.

6. Answer Generation:

The constructed prompt is passed to a generative language model. The model uses the retrieved context to produce a response relevant to the user's question. The generated answer and retrieved source information are then displayed to the user.

The overall RAG workflow is:

$$Documents \rightarrow TextExtraction \rightarrow Chunking$$

$$\rightarrow Embeddings \rightarrow VectorStore$$

$$Question \rightarrow QueryEmbedding \rightarrow Retrieval$$

$$\rightarrow Context + Question \rightarrow LLM \rightarrow Answer$$

7. Advantages of RAG:

RAG allows applications to use domain-specific information without retraining the entire language model whenever documents change. It also makes retrieved sources visible, allowing users to inspect the evidence used to generate an answer. By grounding responses in retrieved context and instructing the model not to invent facts, RAG can reduce unsupported responses.

The system is evaluated using questions whose answers are present in the documents, paraphrased questions, multi-part questions, and questions whose answers are absent from the document collection. This helps assess retrieval relevance and answer reliability.

CODE AND OUTPUT:

Experiment 9 - Retrieval-Augmented Generation (RAG) Document QA

```
!pip -q install sentence-transformers faiss-cpu transformers sentencepiece
```

```
# Small educational corpus based on the public BattingCoach README/project description.
# It summarizes project claims; it is not a substitute for the repository source or a coaching manual.
documents = [
 {"doc_id":"BC01","title":"Project overview","source":"BattingCoach README",
  "text":"BatCoach AI Pro is described as a browser-based cricket batting laboratory and virtual coach. It analyzes webcam video and provides feedback about batting technique."},
 {"doc_id":"BC02","title":"Pose and biomechanics","source":"BattingCoach README",
  "text":"The project describes MediaPipe-based pose estimation and kinematic angle extraction. Its biomechanics module evaluates cues such as lead elbow elevation, lead knee flexion, head-over-knee balance, and spine forward lean."},
 {"doc_id":"BC03","title":"Practice modes","source":"BattingCoach README",
  "text":"The README describes shadow practice without a bat and live willow practice with a bat. Shadow practice bypasses bat-specific checks. Live willow mode includes bat tracking."},
 {"doc_id":"BC04","title":"Computer vision and shot classification","source":"BattingCoach README",
  "text":"The project describes YOLOv8-OBB for oriented bat tracking in live willow mode and a VideoMAE transformer for classifying cricket shots across ten foundational shot types."},
 {"doc_id":"BC05","title":"Feedback and repeated errors","source":"BattingCoach README",
  "text":"The project describes on-screen coaching cues and voice feedback. It tracks total swings and clean repetitions, and repeated technique errors can trigger an intervention that freezes the rep counter."},
 {"doc_id":"BC06","title":"Limitations and responsible use","source":"Notebook-authored evaluation note",
  "text":"Computer-vision feedback depends on camera angle, lighting, occlusion, model confidence and the quality of the input video. A single recording cannot guarantee future match performance or diagnose a medical condition. Treat o
]
chunks = []
for d in documents:
    # short chunks preserve the source title and unique chunk id
    chunks.append({"chunk_id":d["doc_id"]+"-C1","title":d["title"],"source":d["source"],"text":d["text"]})
print("Documents:",len(documents),"Chunks:",len(chunks))
for c in chunks: print(c["chunk_id"],"|",c["title"],"|",c["text"][:90]+"...")
```

```
Documents: 6 Chunks: 6
BC01-C1 | Project overview | BatCoach AI Pro is described as a browser-based cricket batting laboratory and virtual coa...
BC02-C1 | Pose and biomechanics | The project describes MediaPipe-based pose estimation and kinematic angle extraction. Its ...
BC03-C1 | Practice modes | The README describes shadow practice without a bat and live willow practice with a bat. Sh...
BC04-C1 | Computer vision and shot classification | The project describes YOLOv8-OBB for oriented bat tracking in live willow mode and a Video...
BC05-C1 | Feedback and repeated errors | The project describes on-screen coaching cues and voice feedback. It tracks total swings a...
BC06-C1 | Limitations and responsible use | Computer-vision feedback depends on camera angle, lighting, occlusion, model confidence an...
```

```python
import numpy as np, faiss, torch
from sentence_transformers import SentenceTransformer
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

EMBED_MODEL="sentence-transformers/all-MiniLM-L6-v2"
GEN_MODEL="google/flan-t5-small"
embedder=SentenceTransformer(EMBED_MODEL)
device="cuda" if torch.cuda.is_available() else "cpu"
tokenizer=AutoTokenizer.from_pretrained(GEN_MODEL)
llm=AutoModelForSeq2SeqLM.from_pretrained(GEN_MODEL).to(device).eval()

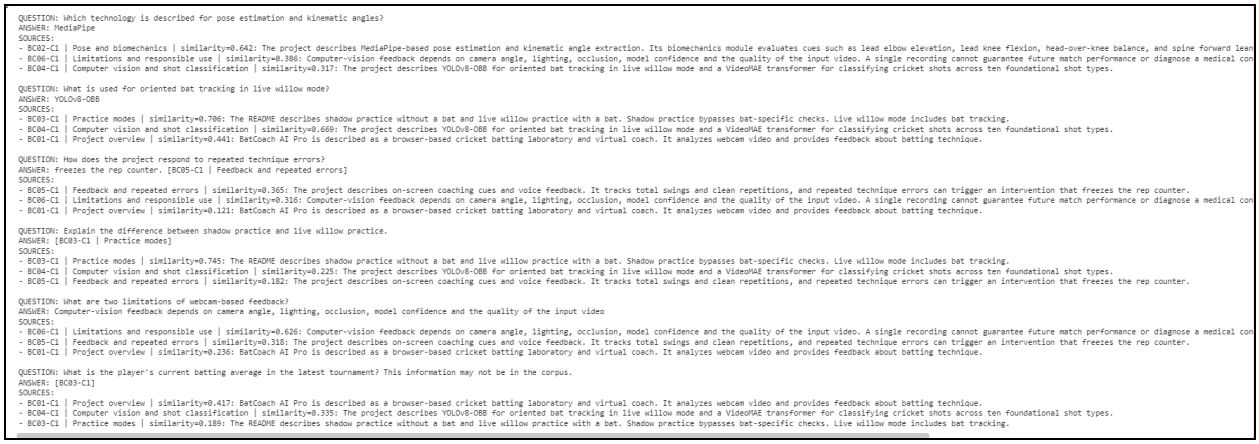
emb=embedder.encode([c["text"] for c in chunks],convert_to_numpy=True,normalize_embeddings=True).astype("float32")
index=faiss.IndexFlatIP(emb.shape[1]); index.add(emb)
print("Indexed",index.ntotal,"chunks. Generator:",GEN_MODEL,"on",device)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads
modules.json: 100% 349/349 [00:00<00:00, 22.1kB/s]
config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 9.33kB/s]
README.md: 100% 10.5k/10.5k [00:00<00:00, 722kB/s]
sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 4.07kB/s]
config.json: 100% 612/612 [00:00<00:00, 55.6kB/s]
model.safetensors: reconstructing file: 100% 90.9MB / 90.9MB, 8.82MB/s
model.safetensors: downloading bytes: 85.0MB, 8.06MB/s
Loading weights: 100% 103/103 [00:00<00:00, 3172.74it/s]
tokenizer_config.json: 100% 350/350 [00:00<00:00, 26.4kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 12.4MB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 21.2MB/s]
special_tokens_map.json: 100% 112/112 [00:00<00:00, 8.23kB/s]
config.json: 100% 190/190 [00:00<00:00, 13.6kB/s]
config.json: 100% 1.40k/1.40k [00:00<00:00, 110kB/s]
tokenizer_config.json: 100% 2.54k/2.54k [00:00<00:00, 195kB/s]
spiece.model: reconstructing file: 100% 792kB / 792kB, 78.9kB/s
spiece.model: downloading bytes: 648kB, 64.7kB/s
```

```python
def retrieve(question, k=3):
    q=embedder.encode([question],convert_to_numpy=True,normalize_embeddings=True).astype("float32")
    scores, ids=index.search(q,k)
    return [(chunks[int(i)],float(s)) for s,i in zip(scores[0],ids[0]) if i >= 0]

def answer_rag(question,k=3):
    retrieved=retrieve(question,k)
    context="\n".join([f"[{c['chunk_id']} | {c['title']}] {c['text']}" for c,score in retrieved])
    prompt=("Answer the question using only the context below. If the context does not contain enough information, "
            "say that the answer cannot be determined from the provided documents. Do not invent facts.\n\n"
            f"CONTEXT:\n{context}\n\nQUESTION: {question}\nANSWER:")
    inputs=tokenizer(prompt,return_tensors="pt",truncation=True,max_length=512).to(device)
    with torch.no_grad():
        out=llm.generate(**inputs,max_new_tokens=120,num_beams=2)
    answer=tokenizer.decode(out[0],skip_special_tokens=True)
    return answer,retrieved

test_questions = [
 "Which technology is described for pose estimation and kinematic angles?",
 "What is used for oriented bat tracking in live willow mode?",
 "How does the project respond to repeated technique errors?",
 "Explain the difference between shadow practice and live willow practice.",
 "What are two limitations of webcam-based feedback?",
 "What is the player's current batting average in the latest tournament? This information may not be in the corpus."
]
rag_records=[]
for q in test_questions:
    ans, ret=answer_rag(q,k=3)
    print("\nQUESTION:",q,"\nANSWER:",ans,"\nSOURCES:")
    for c,score in ret:
        print(f"- {c['chunk_id']} | {c['title']} | similarity={score:.3f}: {c['text']}")
    rag_records.append({"question":q,"answer":ans,"source_chunk_ids":", ".join(c["chunk_id"] for c,s in ret),
                        "retrieved_context":"\n".join(c["text"] for c,s in ret)})
```



```
QUESTION: Which technology is described for pose estimation and kinematic angles?
ANSWER: MediaPipe
SOURCES:
- BC02-C1 | Pose and biomechanics | similarity=0.642: The project describes MediaPipe-based pose estimation and kinematic angle extraction. Its biomechanics module evaluates cues such as lead elbow elevation, lead knee flexion, head-over-knee balance, and spine forward lean
- BC06-C1 | Limitations and responsible use | similarity=0.386: Computer-vision feedback depends on camera angle, lighting, occlusion, model confidence and the quality of the input video. A single recording cannot guarantee future match performance or diagnose a medical con
- BC04-C1 | Computer vision and shot classification | similarity=0.317: The project describes YOLOv8-OBB for oriented bat tracking in live willow mode and a VideoMAE transformer for classifying cricket shots across ten foundational shot types.

QUESTION: What is used for oriented bat tracking in live willow mode?
ANSWER: YOLOv8-OBB
SOURCES:
- BC03-C1 | Practice modes | similarity=0.706: The README describes shadow practice without a bat and live willow practice with a bat. Shadow practice bypasses bat-specific checks. Live willow mode includes bat tracking.
- BC04-C1 | Computer vision and shot classification | similarity=0.669: The project describes YOLOv8-OBB for oriented bat tracking in live willow mode and a VideoMAE transformer for classifying cricket shots across ten foundational shot types.
- BC01-C1 | Project overview | similarity=0.441: BatCoach AI Pro is described as a browser-based cricket batting laboratory and virtual coach. It analyzes webcam video and provides feedback about batting technique.

QUESTION: How does the project respond to repeated technique errors?
ANSWER: freezes the rep counter. [BC05-C1 | Feedback and repeated errors]
SOURCES:
- BC05-C1 | Feedback and repeated errors | similarity=0.365: The project describes on-screen coaching cues and voice feedback. It tracks total swings and clean repetitions, and repeated technique errors can trigger an intervention that freezes the rep counter.
- BC06-C1 | Limitations and responsible use | similarity=0.316: Computer-vision feedback depends on camera angle, lighting, occlusion, model confidence and the quality of the input video. A single recording cannot guarantee future match performance or diagnose a medical con
- BC01-C1 | Project overview | similarity=0.121: BatCoach AI Pro is described as a browser-based cricket batting laboratory and virtual coach. It analyzes webcam video and provides feedback about batting technique.

QUESTION: Explain the difference between shadow practice and live willow practice.
ANSWER: [BC03-C1 | Practice modes]
SOURCES:
- BC03-C1 | Practice modes | similarity=0.745: The README describes shadow practice without a bat and live willow practice with a bat. Shadow practice bypasses bat-specific checks. Live willow mode includes bat tracking.
- BC04-C1 | Computer vision and shot classification | similarity=0.225: The project describes YOLOv8-OBB for oriented bat tracking in live willow mode and a VideoMAE transformer for classifying cricket shots across ten foundational shot types.
- BC05-C1 | Feedback and repeated errors | similarity=0.182: The project describes on-screen coaching cues and voice feedback. It tracks total swings and clean repetitions, and repeated technique errors can trigger an intervention that freezes the rep counter.

QUESTION: What are two limitations of webcam-based feedback?
ANSWER: Computer-vision feedback depends on camera angle, lighting, occlusion, model confidence and the quality of the input video
SOURCES:
- BC06-C1 | Limitations and responsible use | similarity=0.626: Computer-vision feedback depends on camera angle, lighting, occlusion, model confidence and the quality of the input video. A single recording cannot guarantee future match performance or diagnose a medical con
- BC05-C1 | Feedback and repeated errors | similarity=0.318: The project describes on-screen coaching cues and voice feedback. It tracks total swings and clean repetitions, and repeated technique errors can trigger an intervention that freezes the rep counter.
- BC01-C1 | Project overview | similarity=0.236: BatCoach AI Pro is described as a browser-based cricket batting laboratory and virtual coach. It analyzes webcam video and provides feedback about batting technique.

QUESTION: What is the player's current batting average in the latest tournament? This information may not be in the corpus.
ANSWER: [BC03-C1]
SOURCES:
- BC01-C1 | Project overview | similarity=0.417: BatCoach AI Pro is described as a browser-based cricket batting laboratory and virtual coach. It analyzes webcam video and provides feedback about batting technique.
- BC04-C1 | Computer vision and shot classification | similarity=0.335: The project describes YOLOv8-OBB for oriented bat tracking in live willow mode and a VideoMAE transformer for classifying cricket shots across ten foundational shot types.
- BC03-C1 | Practice modes | similarity=0.189: The README describes shadow practice without a bat and live willow practice with a bat. Shadow practice bypasses bat-specific checks. Live willow mode includes bat tracking.
```

```
import pandas as pd
rag_df=pd.DataFrame(rag_records)
display(rag_df)
rag_df.to_csv("experiment9_rag_test_results.csv",index=False)
```

	question	answer	source_chunk_ids	retrieved_context
0	Which technology is described for pose estimat...	MediaPipe	BC02-C1, BC06-C1, BC04-C1	The project describes MediaPipe-based pose est...
1	What is used for oriented bat tracking in live...	YOLOv8-OBB	BC03-C1, BC04-C1, BC01-C1	The README describes shadow practice without a...
2	How does the project respond to repeated techn...	freezes the rep counter. [BC05-C1 | Feedback a...	BC05-C1, BC06-C1, BC01-C1	The project describes on-screen coaching cues ...
3	Explain the difference between shadow practice...	[BC03-C1 | Practice modes]	BC03-C1, BC04-C1, BC05-C1	The README describes shadow practice without a...
4	What are two limitations of webcam-based feedb...	Computer-vision feedback depends on camera ang...	BC06-C1, BC05-C1, BC01-C1	Computer-vision feedback depends on camera ang...
5	What is the player's current batting average i...	[BC03-C1]	BC01-C1, BC04-C1, BC03-C1	BatCoach AI Pro is described as a browser-base...

xt steps: Generate code with rag_df | New interactive sheet

```
# Optional baseline comparison: direct LLM answer without retrieved context.
def answer_direct(question):
    prompt="Answer briefly and state uncertainty if unsure. Question: "+question
    inputs=tokenizer(prompt,return_tensors="pt",truncation=True,max_length=256).to(device)
    with torch.no_grad(): out=llm.generate(**inputs,max_new_tokens=100,num_beams=2)
    return tokenizer.decode(out[0],skip_special_tokens=True)

baseline_q=test_questions[-1]
print("Question:",baseline_q)
print("Direct LLM:",answer_direct(baseline_q))
print("RAG:",answer_rag(baseline_q)[0])
```

```
Question: What is the player's current batting average in the latest tournament? This information may not be in the corpus.
Direct LLM: 69.0
RAG: [BC03-C1]
```

https://colab.research.google.com/drive/1UxmUFVtP_3X4UUsXNaPwkLY-ptW21pcm?usp=sharing

CONCLUSION:

The experiment demonstrated the implementation of a Retrieval-Augmented Generation system for document-based question answering. Documents were processed, divided into chunks, converted into embeddings, and stored in a vector database. Relevant chunks were retrieved for user queries and supplied as context to a generative language model. The system demonstrated how retrieval and generation can be combined to produce more contextually grounded answers and display supporting sources.